UNIVERSITY OF BUEA

P.O Box 63,

Buea, South West Region

CAMEROON

Tel: (237) 3332 21 34/3332 26 90

Fax: (237) 3332 22 72

REPUBLIC OF CAMEROON

PEACE-WORK-FATHERLAND

FACULTY OF ENGINEERING AND TECHNOLOGY

DEPARTMENT OF COMPUTER ENGINEERING AND TECHNOLOGY

DESIGN AND IMPLEMENTATION OF A CAR FAULT DIAGNOSTIC MOBILE APPLICATION

A dissertation submitted to the Department of Computer Engineering, Faculty of Engineering and Technology, University of Buea, in Partial Fulfilment of the Requirements for the Award of Bachelor of Engineering (B.Eng.) Degree in Computer Engineering.

By:

GROUP 21.

Angi Atangwa Valerine	FE22A148
Av'Nyehbeuhkonh Ezekiel Hosana	FE22A160
Beng Dze Meinoff	FE22A173
Jong Turkson Junior	FE22A228
Nyuyesemo Calglain	FE22A287

Option: Software/Network Engineering

Supervisor:

Dr. Nkemeni Valery University of Buea

2024/2025 Academic Year.

Design and Implementation of a Car Fault Diagnostic Mobile Application

Group 21.

Angi Atangwa Valerine	FE22A148
Av'Nyehbeuhkonh Ezekiel Hosana	FE22A160
Beng Dze Meinoff	FE22A173
Jong Turkson Junior	FE22A228
Nyuyesemo Calglain	FE22A287

2024/2025 Academic Year

Dissertation submitted in partial fulfilment of the Requirements for the award of Bachelor of Engineering (B.Eng.) Degree in Computer Engineering.

Department of Computer Engineering

Faculty of Engineering and Technology

University of Buea

Certification of Originality

We the undersigned, hereby certify that this dissertation entitled DESIGN AND IMPLEMENTATION OF A CAR FAULT DIAGNOSIS MOBILE APPLICATION presented by Group 21, has been carried out by him/her in the Department of Computer Engineering, Faculty of Engineering and Technology, University of Buea under the supervision of Dr. Nkemeni Valery

This dissertation is authentic and represents the fruits of their own research and efforts.

Dr. Nkemeni Valery_____

Date_____

DEDICATION

This document is dedicated to **God** almighty **and to** all the members of **Group 21**

ACKNOWLEDGMENT

We the members of group 21 will like to express our gratitude to Dr. Valery Nkemeni for his guidance and support throughout the execution of this project. We also appreciate the contribution of our course mates who provided valuable insights, ideas and review on this project. Additionally we will like to acknowledge the resources and facilities provided by the group21 members and the University of Buea, these resources enabled us to complete the project. Finally we will like to appreciate all the FET lecturers who provided insights and responds to our questions and concerns.

ABSTRACT

Modern vehicles are increasingly complex, and the traditional tools required to detect mechanical faults such as OBD-II scanners are often expensive, hardware-dependent, and inaccessible to everyday users, particularly in developing regions. This project addresses that gap by developing **MFix**, a mobile-based car diagnostics system that provides users with realtime feedback on vehicle faults, without the need for physical diagnostic tools. MFix is designed as a cross-platform application built with **React Native** (via the Expo framework) and powered by **Firebase Firestore**, a cloud-hosted NoSQL database, to ensure real-time data handling, scalability, and ease of use across devices.

The motivation behind MFix is to enhance accessibility and reduce the cost and complexity of vehicle maintenance by allowing users to perform basic fault diagnostics using only their mobile phones. The system supports two user roles regular users and mechanics each with specific functionality. Diagnostics are currently performed using mock data and basic AI-ready structures that simulate the analysis of dashboard images and engine sounds. These features lay the groundwork for future integration of real AI models, trained on labeled datasets, for automated and intelligent fault detection.

The methodology employed includes full-stack development with RESTful APIs, cloud database schema design, role-based authentication, and modular UI components. The backend, implemented using **Node.js** and **Express**, connects securely to Firebase using the Admin SDK, and manages data such as users, diagnostics, results, and requests. The frontend mirrors Figmabased designs and includes lazy loading, state management with React Context API, and error handling simulations.

Results from the implementation phase demonstrate that MFix meets its core objectives of simplicity, scalability, and readiness for AI integration. Real-time data updates, a functional UI for both users and mechanics, and clean data structuring show that the solution is practical and extensible. The app was successfully tested using mock diagnostics and simulated backend calls, ensuring that the infrastructure is solid even before final API integration.

The project contributes to the fields of **mobile computing**, **automotive diagnostics**, and **cloudbased applications**, offering a new approach to car maintenance that is accessible, intelligent, and flexible. Future work will include integration of Bluetooth-based OBD-II scanners, training and deployment of AI models, and the addition of an admin dashboard for analytics and user management.

Keywords: Car diagnostics, React Native, Firebase Firestore, mobile application, AI diagnostics, cloud database, vehicle fault detection.

Table of Contents

List of Tables

List of Abbreviations

1. **OBD-II:** On-Board Diagnostics
2. **AI:** Artificial Intelligence
3. **ML:** Machine Learning
4. **ECU:** Electronic Control Unit
5. **DTC:** Diagnostic Trouble Code
6. **API:** Application Programming Interface
7. **UI:** User Interface
8. **UX:** User Experience
9. **NoSQL:** Non-relational Database
10. **REST:** Representational State Transfer
11. **SDK:** Software Development Kit
12. **MVVM:** Model-View-ViewModel
13. **ERD:** Entity-Relationship Diagram
14. **DFD:** Data Flow Diagram
15. **CI/CD:** Continuous Integration/Continuous Deployment

List of Figures

CHAPTER 1: GENERAL INTRODUCTION

1.1. BACKGROUNG AND CONTEXT OF THE STUDY

In recent years, there has been a rapid increase in private vehicle ownership, especially in developing countries such as Cameroon where road transportation remains the dominant mode of mobility. With this growth comes the rising need for regular vehicle maintenance and efficient fault detection. Vehicles today are built with complex systems and embedded electronics that monitor and control their performance. These systems are capable of displaying warning lights or producing unusual sounds whenever a malfunction is detected. However, many car owners especially those without technical or mechanical knowledge often struggle to interpret these warning signs and take the right action in time.

Traditionally, fault detection has relied heavily on **On-Board Diagnostic (OBD)** tools, which require a physical connection to the vehicle and an external device to detect system faults and alert drivers through **dashboard warning lights**. While effective, these tools are expensive, require technical expertise, and are not readily accessible to ordinary users. Beyond dashboard lights, vehicles also produce **auditory signals** such as knocking, squealing, or rumbling sounds that may indicate underlying problems. However, identifying and interpreting these sounds requires mechanical expertise that the average driver does not possess. In many cases, people must visit a mechanic or diagnostic center just to interpret a simple dashboard light or auditory sound. This delay in response can lead to increased vehicle damage, higher repair costs, and even accidents caused by unnoticed faults.

With the widespread use of smartphones and advancements in artificial intelligence, particularly in image and audio recognition, there is an opportunity to make vehicle diagnostics more **accessible, mobile, and intelligent**. Many car faults can be visually identified (e.g., through dashboard warning icons) or audibly diagnosed (e.g., through engine sounds). Leveraging mobile cameras and microphones, it becomes possible to analyze these inputs using trained AI models, without relying on physical tools or complex hardware.

This study is set within this technological evolution. It explores how **mobile technology, cloud services, and AI-based diagnostics** can be combined to provide a low-cost, user-friendly solution for early fault detection. The development of the **MFix App**—a mobile application designed to scan dashboard images or record engine sounds, analyze them using AI, and suggest

relevant diagnostics and solutions—seeks to close the gap between vehicle fault detection and accessibility for the everyday car owner.

The context of this study is particularly relevant to developing regions such as **Cameroon and many African countries**, where access to diagnostic tools and professional auto-repair services is limited outside major cities. However, mobile phone penetration is high, making this solution both feasible and impactful. By empowering users to understand their vehicle's condition in real time, this project contributes to safer roads, lower repair costs, and increased user confidence in handling vehicle issues.

1.2. PROBLEM STATEMENT

In the current transportation landscape, vehicle users are increasingly dependent on their cars for daily mobility. However, one of the most critical challenges many car owners face is the **inability to detect or interpret early signs of mechanical or electrical faults**. Most modern vehicles are equipped with intelligent dashboard systems that indicate warnings through icons, lights, or unusual sounds. While these indicators are designed to notify users of potential issues, they are often misunderstood or completely ignored due to a lack of technical knowledge.

In many cases, users do not have access to **On-Board Diagnostic (OBD) tools** or the knowledge to operate them. Moreover, visiting a mechanic or service center for every warning light can be **time-consuming, expensive, and sometimes unnecessary**—especially when the problem is minor or easily resolvable. In developing countries like Cameroon, this problem is even more pronounced due to limited access to certified diagnostic services, high vehicle maintenance costs, and a shortage of user-friendly tools tailored to non-technical users.

Additionally, while smartphones are widely available, existing diagnostic apps often require external hardware connections or are not localized for users in low-resource settings. As a result, many vehicle owners are left without a convenient, real-time, and affordable means to identify the cause of dashboard warnings or suspicious engine noises.

This gap between **vehicle fault detection** and **user understanding** often leads to:

- Missed early warnings
- Worsening of small issues into costly damages
- Increased road safety risks
- Overreliance on third-party mechanics even for minor issues

The core problem, therefore, lies in the **lack of an intelligent, accessible, and mobile-based solution that empowers users to diagnose and understand vehicle faults without professional tools**.

1.3. PROPOSED SOLUTION

To address the challenge of inaccessible and unaffordable car fault diagnostics for non-technical users, this project proposes the development of an **AI-powered mobile diagnostic application** named **MFix**. The application will leverage the built-in capabilities of smartphones—namely the **camera** and **microphone**—to allow users to scan their vehicle's dashboard and record engine sounds. These inputs will then be analysed using **computer vision** and **audio classification models** trained on real-world automotive data.

By identifying warning lights and detecting engine sound anomalies, the app will be able to deliver **automated explanations**, **repair suggestions**, and **educational video content** to guide users through possible fixes. The application will also offer **offline functionality** by embedding key symbol detection and sound recognition models, while extended features like video streaming and live database updates will be available when online.

Through this solution, vehicle users will no longer need to rely on physical diagnostic tools or constant visits to auto repair shops for basic issues. Instead, they can gain immediate insight into their vehicle's health, leading to **better maintenance decisions**, **reduced repair costs**, and **enhanced road safety**.

1.4. OBJECTIVES

1.4.1. General Objective

The general objective of this project is to **design and implement an AI powered car diagnostic mobile application,** this app enables car owners to diagnose vehicle faults by recognizing dashboard warning lights and analyzing engine sounds, using only their smartphone's camera and microphone. The goal is to provide a user-friendly, intelligent tool that delivers real-time explanations, suggested fixes, and video tutorials to assist in preventive vehicle maintenance.

1.4.2. Specific Objectives

To achieve the above goal, the project focuses on the following specific objectives:

- **To develop a mobile application** using React Native (Expo) that allows users to scan their vehicle's dashboard and record engine sounds directly from their smartphone.
- **To implement computer vision techniques** capable of detecting and recognizing multiple dashboard warning lights from captured images, even when several indicators are active at once.
- **To integrate machine learning or deep learning models** that can analyze recorded engine sounds and classify them against known fault types such as knocking, squealing, or hissing.
- **To provide automatic explanations** of detected warning symbols, including their causes, urgency level, and consequences of neglect.
- **To generate a list of likely mechanical issues** along with suggested repair actions, general maintenance advice, and urgency levels.

- **To integrate YouTube Data API v3** to fetch video tutorials from verified automotive experts, helping users understand step-by-step maintenance procedures for diagnosed issues.
- **To ensure basic offline functionality** by embedding pre-trained diagnostic models and symbol data for use without an internet connection.
- **To enable online capabilities** such as accessing the latest fault databases, retrieving updated explanations, and streaming or embedding relevant video content.

1.5. PROPOSED METHODOLOGY

To achieve the stated objectives, this study adopts a modular and iterative methodology combining mobile development, cloud-based services, and AI integration. The approach is structured into logical phases that support continuous testing, validation, and refinement of each system component. Each phase focuses on implementing specific features of the MFix diagnostic application in alignment with the user's needs and the system's functional requirements.

1.5.1. System Design and Planning

The project begins with the definition of functional and non-functional requirements, identification of core use cases, and development of user interface designs using **Figma**. These designs follow mobile UI/UX principles and reflect the user's interaction flow. A high-level system architecture is then designed, outlining the integration of the frontend (mobile app), backend server, AI services, and database components.

1.5.2. Frontend Development of the Mobile Application

The frontend is implemented using **React Native with Expo**, allowing the app to run on both Android and iOS platforms with a single codebase. This phase includes:

- Enabling access to the device **camera** for dashboard scanning.

- Enabling access to the **microphone** to record engine sounds.
- Developing key screens such as onboarding, scanning interface, diagnostic result pages, and video tutorial viewers.
- Ensuring **basic offline functionality** by embedding local data and fallback logic when the device is not connected to the internet.

1.5.3. Backend Development and Database Integration

The backend is developed using **firebase**, connected to **Firebase services** through the **Firebase Admin SDK**. Key activities include:

- Configuring **Firebase Firestore** as the NoSQL database to store user data, diagnostics, and results.
- Implementing **Firebase Authentication** to manage user login, registration, and rolebased access control.
- Integrating **Firebase Cloud Storage** for secure upload and retrieval of diagnostic images and engine recordings.
- Creating secure API routes to handle data transmission between the mobile app and backend.

1.5.4. Artificial Intelligence Integration

The diagnostic functionality of the application relies on AI models capable of interpreting visual and audio inputs. This includes:

- Using the **Gemini API** or equivalent AI models to analyze dashboard images and identify active warning lights through computer vision.
- Implementing or integrating **pre-trained audio classification models** to detect patterns in engine sound recordings and match them to known fault categories (e.g., knocking, hissing).

- Generating automated explanations, urgency levels, and suggested repair actions based on AI responses.

1.5.5. Educational Video Integration

To enhance user understanding, the application integrates the **YouTube Data API v3** to retrieve relevant video tutorials. These tutorials are:

- Matched dynamically to identified fault types.
- Filtered by reliability and source authenticity (from verified automotive content creators).

1.5.6. Testing and Validation

A comprehensive testing process is adopted at multiple stages:

- **Unit Testing**: Performed on individual modules such as image capture, sound recording, and result parsing.
- **API Testing**: Using **Postman** to verify backend endpoints and responses.
- **Integration Testing**: Simulating full diagnostic flows from scanning/recording to result display and video guidance.
- **User Testing**: Conducted with non-technical users to assess ease of use, clarity of results, and overall experience.

1.5.7. Deployment and Demonstration

Upon completion, the system will be:

- Deployed using **Firebase Functions** and/or a hosted backend server.
- Packaged as an installable mobile application.
- Demonstrated to simulate real-world usage: capturing dashboard images, recording engine sound, and receiving diagnostic feedback along with video tutorials.

This structured methodology ensures that the system is not only functional and user-friendly but also robust, scalable, and aligned with modern mobile and AI development practices.

1.6. RESEARCH QUESTION

This study seeks to explore how artificial intelligence and mobile technology can be integrated to provide a practical, user-friendly vehicle diagnostic tool for non-technical car owners. The following research questions guide the direction of the investigation and the evaluation of the proposed solution:

- How effectively can artificial intelligence be used to detect and interpret multiple dashboard warning lights from images captured by a smartphone camera?
- Can audio classification models accurately identify mechanical faults based on recorded engine sounds using a mobile device?
- To what extent can a mobile application reduce car owners' reliance on traditional OBD hardware and professional diagnostic tools?
- Does integrating AI-generated explanations and video tutorials improve users' understanding of vehicle issues and their ability to take timely maintenance actions?
- Can a mobile-based diagnostic app provide reliable offline and online functionalities that meet the needs of car owners in low-resource or developing regions?

These research questions form the foundation of the project's design and implementation, guiding both the system's technical architecture and the evaluation criteria for its success.

1.7. RESEARCH HYPOTHESIS

A hypothesis is a statement that predicts the outcome of a study based on existing knowledge, theory, or observation. In the context of this project, the hypotheses aim to validate the assumption that artificial intelligence, when integrated into a mobile platform, can enable effective and accessible vehicle diagnostics for non-technical users.

The research is guided by the following hypotheses:

1.7.1. Main Hypothesis (H_1)

H_1: Artificial intelligence can accurately recognize dashboard warning lights and analyze engine sounds through mobile devices, providing effective diagnostic information to car owners without the need for physical diagnostic hardware.

1.7.2. Supporting Hypotheses

- **H_2:** A mobile application that combines image and sound analysis can detect multiple types of vehicle faults with acceptable accuracy compared to traditional tools.
- **H_3:** The integration of AI-generated explanations and video tutorials enhances user understanding and response to vehicle faults.
- **H_4:** Mobile-based diagnostics can significantly reduce the frequency of unnecessary visits to professional auto repair shops for minor or non-urgent issues.
- **H_5:** The use of offline AI models can provide basic diagnostic capabilities even in areas with limited internet access.

These hypotheses will be tested through system implementation, data collection, and performance evaluation during the development and user testing phases.

1.8. SIGNIFICANCE OF THE STUDY

The significance of this study lies in its potential to transform how vehicle owners interact with their cars and respond to faults. In many parts of the world—especially in developing countries like Cameroon—access to professional diagnostic tools, such as OBD scanners, is limited by cost, availability, and technical expertise. As a result, many vehicle users fail to act promptly when dashboard warning lights appear or when abnormal engine sounds are noticed, often leading to serious breakdowns, increased repair costs, and safety hazards.

This study is important because it offers a **practical, intelligent, and accessible solution** to this common challenge. By designing a mobile application that utilizes **artificial intelligence (AI)** to interpret dashboard symbols and analyze engine sounds, the project significantly reduces the user's dependence on professional mechanics for basic diagnostics. It empowers non-technical users to understand their vehicles in real time, improving maintenance habits and promoting road safety.

Additionally, the integration of **educational video content** from trusted automotive experts transforms the app into not only a diagnostic tool but also a learning platform. This encourages preventive maintenance and increases user confidence in handling minor mechanical issues without unnecessary visits to repair shops.

From a technological perspective, this study demonstrates the practical application of AI and mobile computing in the automotive sector. It shows how **computer vision, sound classification, and cloud services** can be combined in a mobile-first solution to solve realworld problems. Academically, the project serves as a model for future research in mobile diagnostics, AI-assisted maintenance, and smart mobility systems.

In summary, this study contributes to:

- **Improved accessibility** to vehicle diagnostics,
- **Cost-effective maintenance** for car owners,
- **Technological innovation** in AI-integrated mobile apps, and
- **Local development**, especially in contexts with limited technical infrastructure.

1.9. SCOPE OF THE STUDY

This study focuses on the design and implementation of **MFix**, a mobile-based vehicle diagnostic application that empowers car owners to identify and understand vehicle faults using artificial intelligence and smartphone camera/microphone. The application is built to simulate essential diagnostic features such as dashboard icon recognition and engine sound analysis —

providing a cost-effective and accessible alternative to traditional vehicle fault diagnosis systems.

The MFix app primarily targets car owners who lack access to advanced diagnostic tools or professional mechanical expertise. It uses a **smartphone camera** to scan dashboard indicators and a **microphone** to record engine sounds, which are then analyzed using AI models to detect abnormalities and recommend possible actions. The system provides **real-time explanations**, **video-based repair guidance**, and **maintenance suggestions** to support early fault detection and preventive maintenance.

While traditional diagnostic tools often rely on direct connection to a vehicle's **OBD-II port** via Bluetooth or Wi-Fi to retrieve **Diagnostic Trouble Codes (DTCs)** and real-time engine parameters, this study does **not** implement direct OBD integration. Instead, it proposes an alternative that **mimics key OBD functionalities** using AI and accessible device inputs. Features such as **fault history logs**, **basic offline analysis**, **image/sound-based fault identification**, and **video tutorials** are developed to support users in low-resource contexts where physical OBD scanners are unavailable or unaffordable.

Specifically, the scope of this project includes:

- Development of a **cross-platform mobile application** using React Native (Expo).
- AI-powered **dashboard light recognition** through computer vision models.
- AI-driven **engine sound analysis** to detect common fault signatures.
- Integration with **Firebase Firestore** for data storage and user management.
- Use of **YouTube Data API** to recommend verified repair tutorials.
- Inclusion of **offline capabilities** for basic diagnostics using pre-trained models.
- Focus on the **end-user experience**, emphasizing ease of use for non-technical users.

However, the study does **not** cover:

- Real-time vehicle telemetry using OBD-II hardware.
- Integration with manufacturer-specific diagnostic protocols or sensors.
- Predictive analytics based on long-term vehicle performance trends.

The application is primarily designed for deployment in **developing regions**, such as Cameroon, where access to diagnostic infrastructure is limited but smartphone usage is widespread.

1.10. DELIMITATION OF THE STUDY

To ensure focused development and realistic implementation within the timeframe and resources available, several **delimitations** were defined for this study. These decisions helped maintain technical feasibility while concentrating on the core objectives of mobile-based vehicle fault diagnosis using AI.

The study is delimited in the following ways:

- The system does **not use actual OBD-II hardware** or direct vehicle data from the ECU. Although OBD integration is a common standard for vehicle diagnostics, the project focuses instead on **camera-based** and **audio-based** analysis as a more accessible alternative for everyday users, especially in low-resource settings.
- The AI models used for dashboard symbol recognition and sound classification are trained on **a limited dataset** curated for commonly occurring fault types. The app does not yet support a full range of vehicle makes or proprietary error codes.
- The application is **designed for Android and iOS smartphones only**. It does not support web, desktop, or in-vehicle embedded systems.
- The app provides fault detection and suggested maintenance guidance for **informational purposes only**. It does not replace professional mechanical diagnostics or repairs, and it is not certified by any automobile regulatory authority.
- The project is delimited to **common dashboard icons** and **engine fault sounds**. It does not include advanced vehicle health monitoring features such as fluid level sensing, tire pressure monitoring, or real-time telemetry data.
- While online features such as video tutorials and cloud updates are available, the app's **offline functionality is limited** to preloaded recognition data. Real-time AI inference in offline mode is constrained to the device’s local processing power.

- The application targets **individual car owners and drivers**. It is not designed for fleet management systems or commercial garage operations.

By setting these delimitations, the study remains focused on delivering a **functional, accessible, and intelligent diagnostic tool** for everyday vehicle users, particularly those without access to traditional hardware or repair infrastructure.

1.11. DEFINITION OF KEYWORDS AND TERMS

Table 1: Definition of keywords and terms

Term / Keyword	Definition
Car Diagnostics	The process of identifying and analyzing faults or issues in a vehicle's systems, typically using sensors, scanners, or AI-based models.
Term / Keyword	**Definition**
Mobile Application	A software program designed to run on smartphones and tablets. In this project, the mobile app is developed using React Native for crossplatform use.
React Native	A JavaScript-based framework used to build native mobile applications for Android and iOS from a single codebase.
Firebase Firestore	A scalable NoSQL cloud database that enables realtime data storage and synchronization between mobile clients and the backend.
Backend	The server-side part of an application that handles logic, data processing, and communication between the database and frontend.
Frontend	The client-side interface that users interact with, typically designed with user experience and design principles in mind.
Expo	A toolchain built around React Native that simplifies mobile development, testing, and deployment.

Mechanic	A user role in the system responsible for providing diagnostic services or feedback to vehicle owners.
User	A general system role for individuals who register their vehicles and initiate diagnostic requests.
Diagnostics Result	The outcome of a simulated or AI-powered analysis of a vehicle image or sound, including identified faults and recommendations.
NoSQL Database	A non-relational database that stores data in documents or collections rather than rows and columns, allowing for flexibility in data structure.
Mock Data	Simulated or hardcoded data used in place of real API responses to enable frontend testing and interface development before full backend integration.
Context API	A method in React for managing and sharing state across components without prop drilling.
AsyncStorage	A persistent key-value storage system in React Native used to save small amounts of data locally on the user's device.
Machine Learning (ML)	A subset of artificial intelligence that allows systems to learn patterns from data and make decisions or predictions without explicit programming.
Term / Keyword	**Definition**
Hugging Face	An online platform that hosts pre-trained AI models, particularly in natural language processing and audio/image recognition.
Kaggle	A data science platform that provides datasets and competitions, often used for training and testing ML models.
Term / Keyword	Definition
Car Diagnostics	The process of identifying and analyzing faults or issues in a vehicle's systems, typically using sensors, scanners, or AI-based models.
Mobile Application	A software program designed to run on smartphones and tablets. In this project, the mobile app is developed using React Native for crossplatform use.

React Native	A JavaScript-based framework used to build native mobile applications for Android and iOS from a single codebase.
Firebase Firestore	A scalable NoSQL cloud database that enables realtime data storage and synchronization between mobile clients and the backend.
Backend	The server-side part of an application that handles logic, data processing, and communication between the database and frontend.
Frontend	The client-side interface th at users interact with, typically designed with user experience and design principles in mind.
Expo	A toolchain built around React Native that simplifies mobile development, testing, and deployment.
Mechanic	A user role in the system responsible for prov iding diagnostic services or feedback to vehicle owners.
User	A general system role for individuals who register their vehicles and initiate diagnostic requests.
Diagnostics Result	The outcome of a simulated or AI-powered analysis of a vehicle image or sound, including identified faults and recommendations.
NoSQL Database	A non-relational database that stores data in documents or collections rather than rows and columns, allowing for flexibility in data structure.
Mock Data	Simulated or hardcoded data used in place of real API responses to enable frontend testing and interface development before full backend integration.
Term / Keyword	**Definition**
Context API	A method in React for managing and sharing state across components without prop drilling.
AsyncStorage	A persistent key-value storage system in React Native used to save small amounts of data locally on the user's device.
Machine Learning (ML)	A subset of artificial intelligence that allows systems to learn patterns from data and make decisions or predictions without explicit programming.

Hugging Face	An online platform that hosts pre-trained AI models, particularly in natural language processing and audio/image recognition.
Kaggle	A data science platform that provides datasets and competitions, often used for training and testing ML models.

1.12. ORGANISATION OF THE DISSERTATION

This dissertation is organized into five main chapters, each serving a distinct purpose in presenting the research, development, and findings of the project.

- **Chapter One** provides a general introduction to the study. It outlines the background and context, problem statement, objectives, proposed solution, methodology, research questions, and other foundational elements of the project.
- **Chapter Two** presents a review of relevant literature, highlighting existing solutions, related technologies, and theoretical foundations that inform the design and development of the proposed system.
- **Chapter Three** describes the system design, including the architectural framework, database design, user interface models, and the overall technical approach used to build the application.
- **Chapter Four** discusses the actual implementation of the system, detailing the technologies used, backend integration, AI functionality, testing procedures, and results obtained.
- **Chapter Five** concludes the study by summarizing key achievements, identifying challenges encountered, and providing recommendations for future work and potential improvements.

This chapter provided a comprehensive overview of the motivation, problem context, proposed methodology, and scope of the MFix project. It also outlined the key research questions and hypotheses that guide the development of the system. The following

chapter presents a critical review of existing works, tools, and technologies related to mobile diagnostics, AI integration, and automotive fault detection.

2. CHAPTER 2: LITERATURE REVIEW

2.1. INTRODUCTION

This chapter presents a review of existing literature related to vehicle fault diagnostics, mobile diagnostic applications. It aims to provide the theoretical foundation and identify gaps in current systems that justify the development of MFix — a mobile-based car fault diagnostic app. The review encompasses general concepts like On-Board Diagnostics (OBD-II), sound and imagebased fault detection, and the use of mobile platforms for car health monitoring.

2.2. GENERAL CONCEPTS ON VEHICLE FAULTS DIAGNOSTIC APPLICATIONS

2.2.1. On-Board Diagnostics (OBD-II)

The OBD-II system is a standardized protocol used in modern vehicles for self-diagnostics and reporting. It provides access to various vehicle subsystems for monitoring engine health, fuel consumption, emissions, and more. Using an OBD-II scanner, diagnostic trouble codes (DTCs) can be retrieved and interpreted.

According to Tracy Martin (2012), the OBD-II interface allows external devices to communicate with the vehicle's electronic control unit (ECU) via protocols like SAE J1850 or ISO 9141. These interfaces form the technical foundation upon which diagnostic apps like MFix operate.

2.2.2. Fault Codes and Diagnostics

Fault codes are numeric codes that point to specific issues in the vehicle, ranging from minor sensor errors to critical engine malfunctions. These codes are classified into standard and manufacturer-specific types. Accurate interpretation of these codes is essential for effective diagnostics and maintenance planning.

2.2.3. Monitoring

Modern diagnostics go beyond code reading to support real-time monitoring of live sensor data such as RPM, throttle position, and fuel trims. Real-time capabilities are crucial for identifying intermittent faults and trends in vehicle performance.

2.2.4. Mobile Diagnostic Applications

Mobile applications have revolutionized car diagnostics by making tools accessible and affordable. Using Bluetooth or Wi-Fi OBD-II adapters, smartphones can now act as diagnostic tools. These apps often display data graphically, suggest fixes, and allow fault code clearing.

Apps such as Torque and OBDLink have shown how mobile interfaces can simplify diagnostics for both experts and lay users (Smith et al., 2017). However, these apps often lack advanced features like AI-based predictions or image/sound diagnostics, which are part of MFix's innovation scope.

2.3.RELATED WORKS

2.3.1. Torque Pro

Torque is an Android-based app that connects to a vehicle's OBD-II system via Bluetooth. It allows users to read fault codes, clear them, and view real-time data. Its strengths lie in ease of use and dashboard customization. However, it lacks predictive analytics and support for nontextual input like sound.

Fig1: Torque App

2.3.2. FIXD

FIXD provides a consumer-friendly interface that explains fault codes in simple terms. It targets everyday drivers and includes maintenance reminders. While accessible, FIXD has limited diagnostic depth compared to more advanced tools. FIXD requires a sensor and a cable

Figure 2: FIXD preinstallation on google playstore

2.3.3. Research on Mobile Diagnostic Systems

In their paper, "Development of a Mobile-Based Vehicle Fault Diagnosis System Using OBD-II," Smith et al. (2017) highlight how Android platforms can be leveraged to deliver portable diagnostic solutions. The study emphasizes Bluetooth-based communication with OBD-II but notes challenges such as limited sensor access and compatibility issues.

AI and Sound/Image-based Diagnostics

K. Patel (2020) explored how machine learning can be applied to sensor data for predictive maintenance. Their study used classification algorithms on sensor readings to identify patterns linked to known faults.

Garcia et al. (2021) reviewed how sound analysis (e.g., from engine noise) can detect faults that may not be captured by sensors. This suggests that incorporating microphones into mobile diagnostic apps could improve early detection.

Image processing also plays a role in vehicle diagnostics. For instance, visual inspection systems using cameras can detect damage or wear, which aligns with MFix's future development scope.

2.3.4. Limitations in Existing Solutions

Current applications suffer from multiple limitations:

- Limited support for newer vehicle models or proprietary manufacturer protocols.
- Poor integration of advanced features like sound/image analysis.
- Dependence on constant internet access.
- Weak user interface design for non-experts.

These gaps provide justification for developing MFix — a modern, modular, and AI-augmented diagnostic app built using React Native for cross-platform compatibility.

2.4.PARTIAL CONCLUSION

This chapter has reviewed key literature on OBD-II systems, mobile diagnostics, and the integration of AI technologies in vehicle health monitoring. While numerous mobile apps and systems exist, they are often limited in functionality or accessibility. The MFix app aims to address these gaps by offering a user-friendly interface, modular diagnostic capabilities, and support for sound/image input in future versions. The next chapter will discuss the methodology used in designing and developing the MFix system.

3. CHAPTER 3: ANALYSIS AND DESIGN

3.1.INTRODUCTION

This chapter presents the detailed analysis and design of the Car Fault Diagnostic App (**MFix**), developed to assist vehicle owners in identifying faults through AI-driven image and sound recognition. The design is informed by the findings in the literature review and the problem analysis outlined in earlier chapters. It includes the adopted methodology, system architecture, UI design decisions, component breakdowns, and visual models that guided implementation. The chapter also details the tools and technologies used for the design and how various modules interact to achieve the system's objectives.

3.2.PROPOSED METHODOLOGY

The design and analysis of the MFix system followed a structured, iterative approach inspired by the **Agile methodology**, which emphasizes adaptability, user feedback, and incremental refinement. Although full implementation was covered in later phases, the focus of this chapter is on how the system was conceptualized, modelled, and structured before development.

- **Design Thinking Approach**

To guide the analysis and design process, the team adopted a **user-centered design approach**, which places the end-user's needs, behaviors, and constraints at the core of all design decisions.

The methodology followed the logical progression outlined below:

3.2.1. Requirements Gathering

The first phase involved identifying and understanding the problem faced by everyday car users, particularly their inability to interpret dashboard warnings and engine issues without professional assistance. To gather relevant requirements:

- **Observation** and **informal interviews** were conducted with drivers and car owners.

- **Survey forms** were distributed to various car owners and drivers.
- **Questionnaires** were used to collect data on the most common car fault challenges based on mechanics experience.
- Research from **Chapter 2 (Literature Review)** helped identify functional gaps in existing diagnostic apps.

3.2.2. Requirement Analysis and Prioritization

The collected data was analyzed to extract functional and non-functional requirements. These were categorized and prioritized based on:

- Frequency of user need
- Feasibility within the academic timeline
- Technical complexity

This analysis led to a clear list of **core requirements** such as:

- Dashboard image scanning
- Engine sound analysis
- Fault explanations
- Repair video integration
- Offline functionality

3.2.3. System Design Phase

With the key requirements established, the next phase focused on designing the overall system behavior and structure. This included:

- **Use case diagrams** to model interactions between the system and different users

- **Activity and sequence diagrams** to represent workflows and process logic
- **Entity-relationship diagrams (ERDs)** for conceptual data modeling
- A clear **modular breakdown** of system components (frontend, backend, AI modules, data storage)

All of these elements were documented in the **Software Design Document (SDD)**.

3.2.4. User Interface Design

Once the functional modules were mapped out, attention was given to the user interface and experience:

- **Wireframes** were hand-sketched to model screen layouts and user navigation.
- After approval, high-fidelity **mockups were created in Figma**, simulating each app screen from login to results display.
- The UI design followed principles of accessibility, consistency, and minimalism — ensuring clarity even for non-technical users.

This design-focused methodology ensured that every feature in MFix was carefully planned, documented, and validated at the analysis stage before any implementation began.

3.3.DESIGN

The design phase focused on transforming system requirements into technical blueprints that guided implementation. The system was divided into frontend, backend, and database components, with a focus on modularity, scalability, and real-time interaction.

Key design objectives included:

- Cross-platform mobile app with a user-friendly interface.
- Role-based access control (user vs. mechanic).
- Real-time data interaction via Firebase.
- Component-based architecture for maintainability.

System Design Artifacts:

- **Use Case Diagrams**: Defined user interactions like registration, diagnosis request, result viewing.

It illustrates what the system will do and not how it will do it. Each interaction is shown as a "use case," which describes a single function or goal that the system must support.
The actors of our system are separated into three categories, the primary actor in this case it is the user who is probably a car owner, secondary actors here the system it self and tertiary actor has little within the system or interacts indirectly with the system in this case it is the mechanic and machine learning models. But in our diagram only the mechanic is present and represented as the tertiary actor.

The diagram below illustrates our usecase diagram showing the relationship that exists betwwen the actors and the usecases and the between the various usecases.

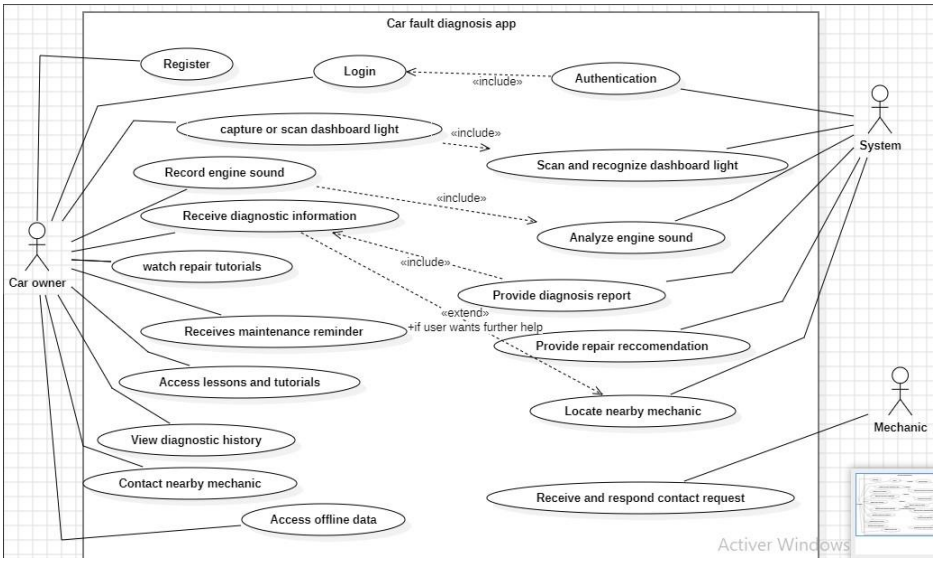


Fig3: UseCasse Diagram

- **Sequence Diagrams**: Modeled request flow from frontend to backend and database.

Below is the sequence diagram of the car fault diagnostic app showing each sequence and its description **Sequence diagrams 1. Login:**

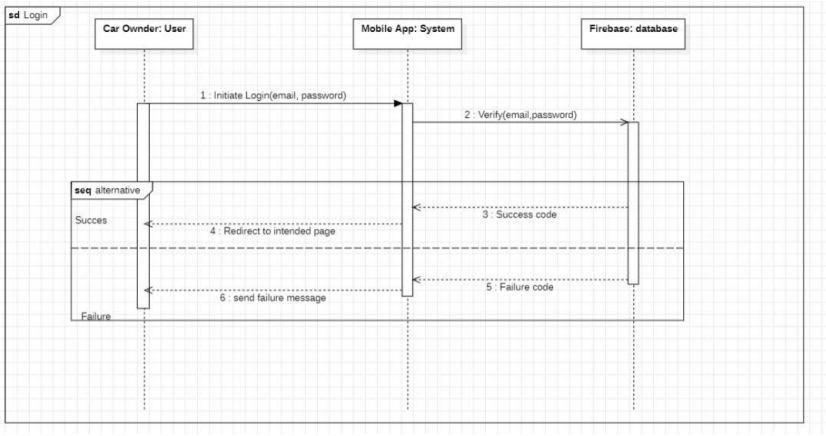


Fig4 : sequence diagram Login process 2.

Create account:

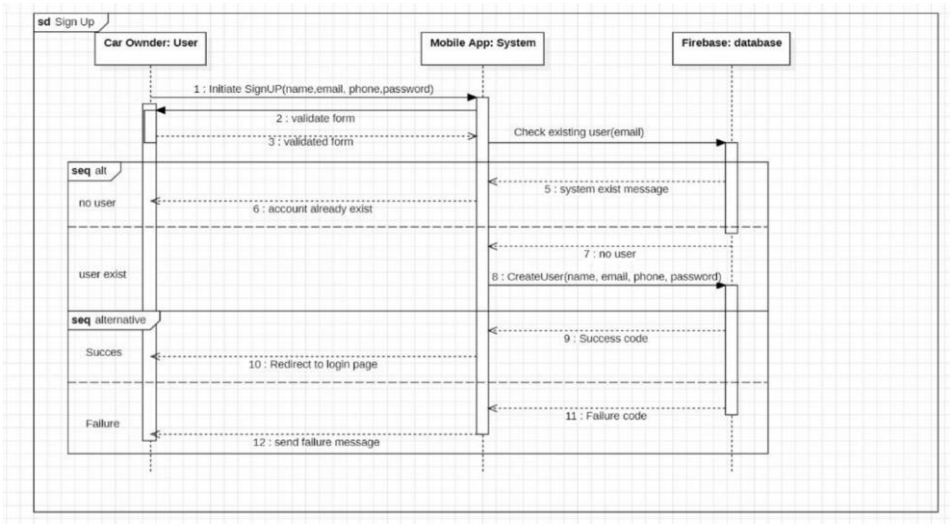


Fig5: sequence diagram, create account

3. Dashboard light scanner

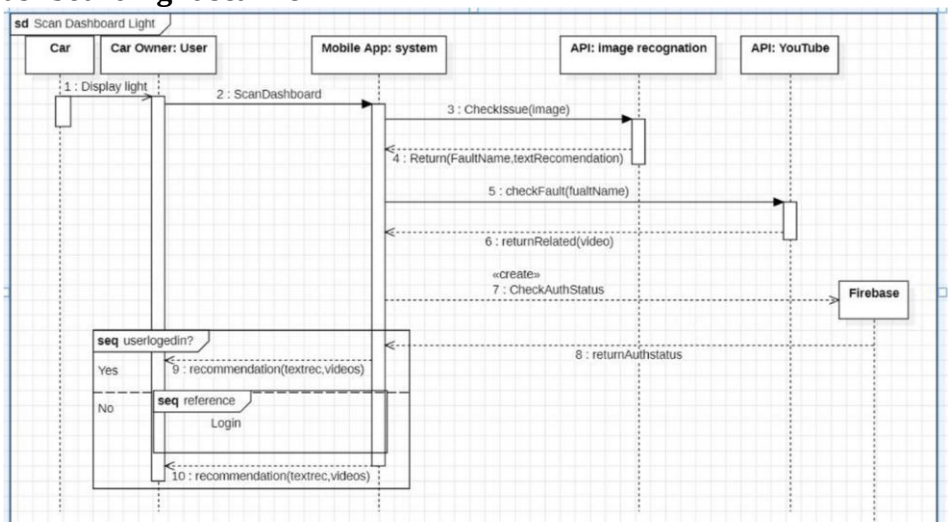


Fig6: Sequence diagram dashboard scan

4. Engine sound diagnostics.

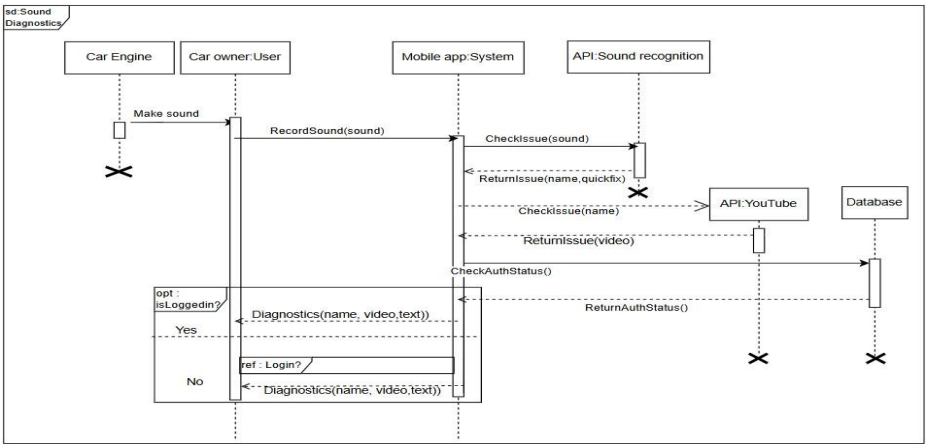


Fig7: sequence diagram engine sound diagnosis

5. View recommendations

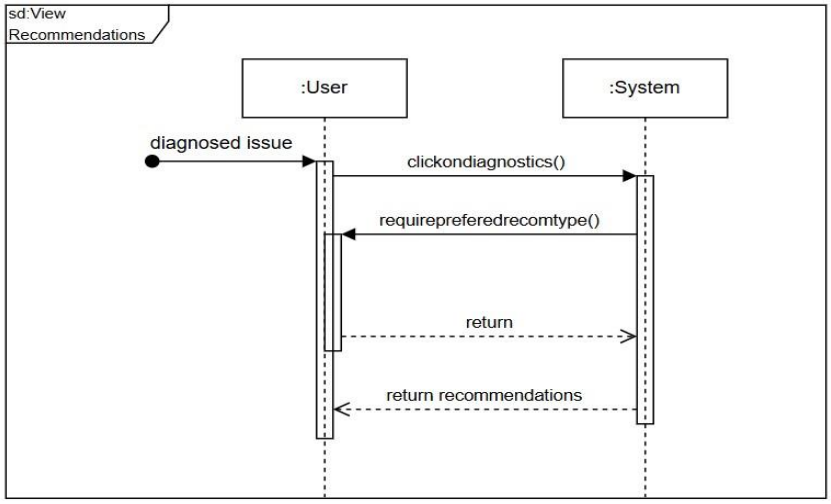


Fig8: sequence diagram view reccommendation

- **Data Flow Diagrams**: Illustrated how diagnostic data and user actions flow across components.

DFDs typically have four key components:

- **Processes**: Represented by rounded rectangles or circles; show transformations of data.
- **Data Stores:** Represented by open-ended rectangles; show where data is stored.

- **External Entities**: Represented by squares; indicate sources or destinations of data outside the system.

- **Data Flows**: Represented by arrows; indicate the movement of data between components.Our system dataflow diagram included six main processes which include,

1.1.0**Authentication**

2. 2.0 **Fault Diagnosis (Sound and Dashboard)** -

3. 3.0 **Repair Recommendation**

4. 4.0 **Track Maintenance**

5. 5.0 **Mechanic Request**

6. 6.0 **Tutorial Provision**

- The diagram below represents our data flow level1 diagram

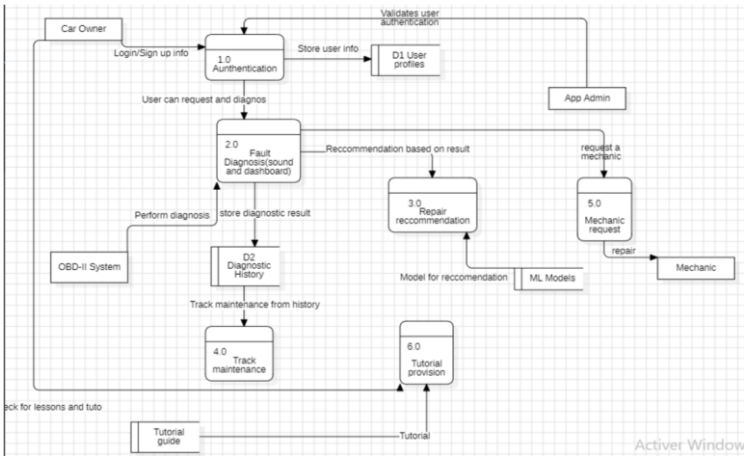


Fig9: Dataflow diagram

- **Context diagram**

Purpose of the context diagram:

- To establish **boundaries** of the system
- To make sure the system align with its Scope
- To provide a **simple, easy-to-understand** visual overview of system interactions

Below is the context diagram which illustrates the system boundaries and interactions with external entity

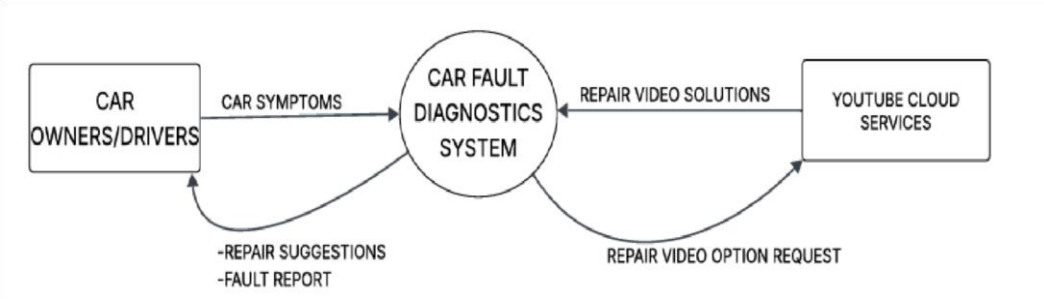


fig10: context diagram

- **Class diagram**

Class diagrams provide a high-level view of a system's architecture and are especially useful during software design and documentation. The key components include:

- **Classes:** Represent entities in the system, containing attributes and methods.
- **Attributes:** Define the data held by a class.
- **Methods**: Represent behaviors or functions a class can perform.
- **Relationships:** Show how classes are associated with each other (e.g., association, inheritance, aggregation).

Our class diagram models a Car Fault Diagnostics System with the following core components:

User and CarOwner:

Vehicle and Maintenance:

`MaintenanceRecord`.

Diagnostics:

Mechanic and Garage:

Notification and Tutorials:

- **`Notification`** handles sending updates to users and logs delivery status.
- **`RepairTutorial`** is associated with **`DiagnosticResult`**, offering tutorial content and video playback functionality.

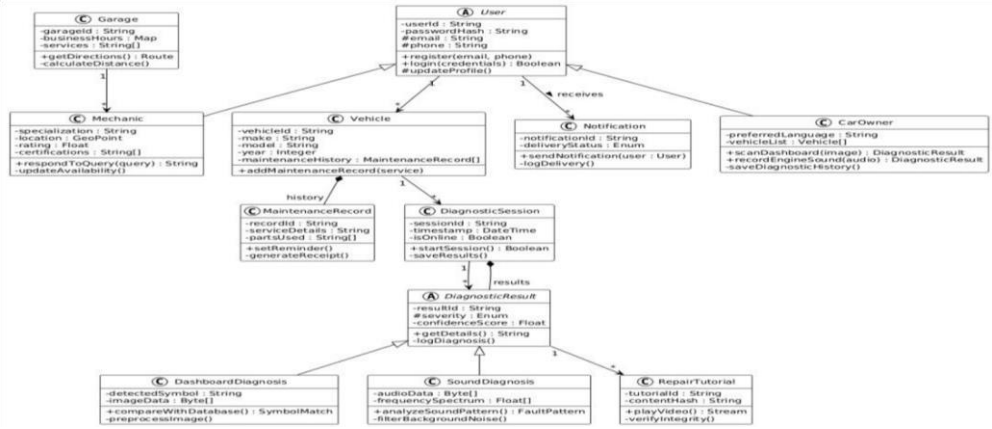


Fig11: Class Diagram

- **Deployment diagram**

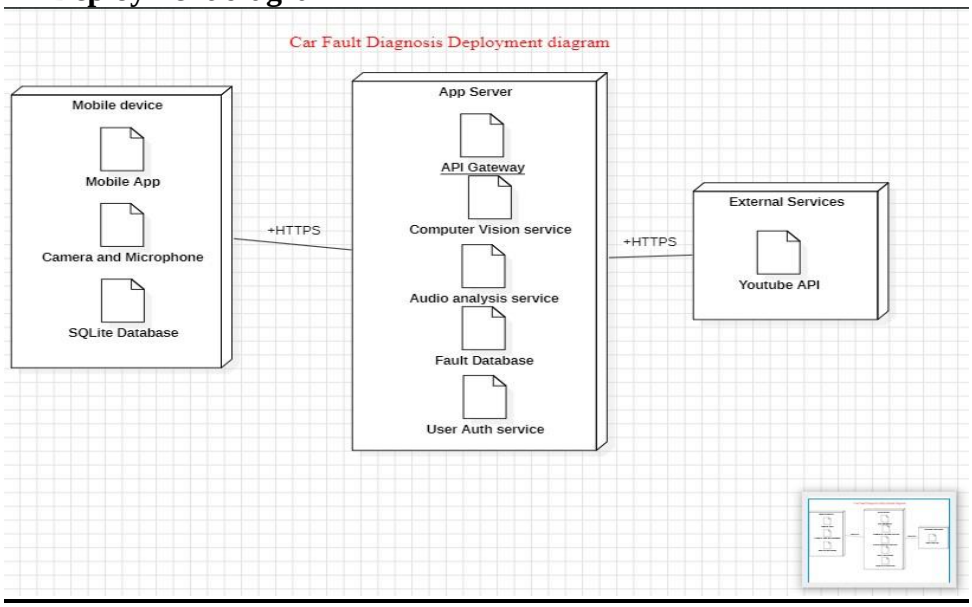


Fig12: deployment diagram

- **ER Diagram / Firestore Schema**: Logical structure of collections, documents, and relationships.

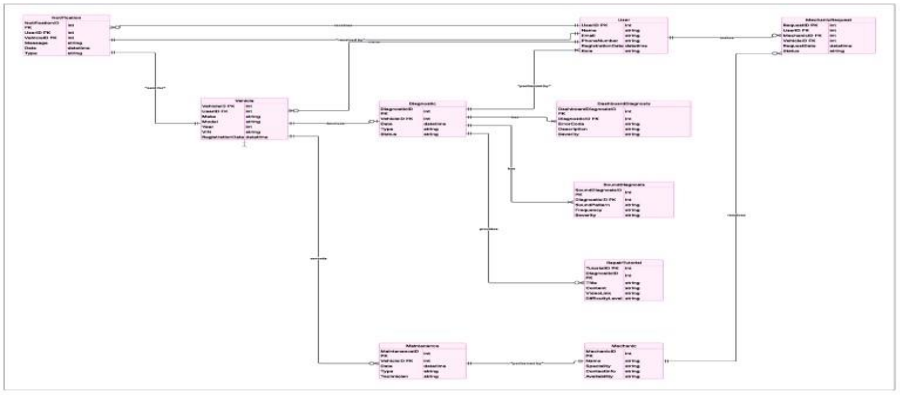


fig13: ER diagram

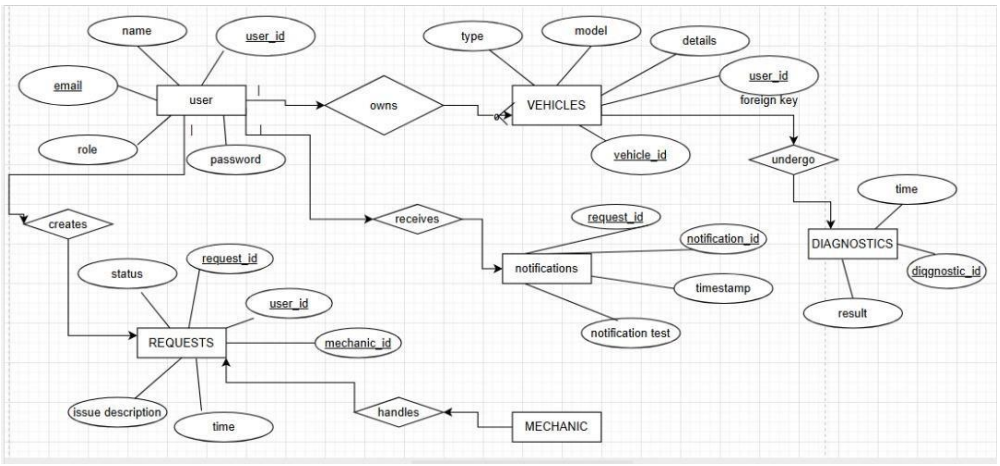
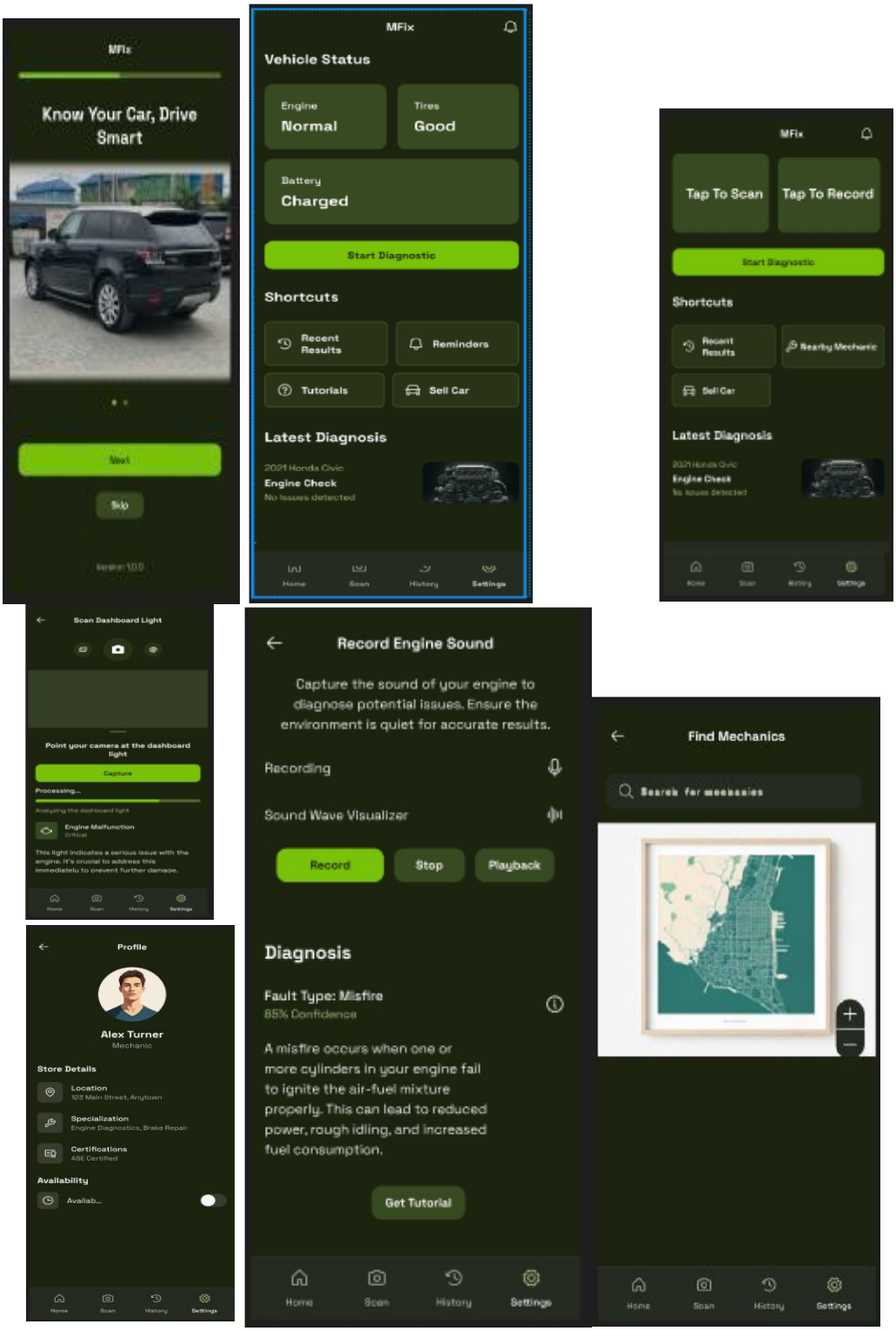


Fig14: conceptual diagram

- After the system model and diagrams where done, we went further to give a brand and identity to the car fault diagnosis mobile app. We named the application MFix where 'M' there stands for Motto (vehicle in most Cameroon native languages) and 'Fix' for repairing. We also determined a colour system of Green, white and black and designed a logo based on that. After the brand identity and colour system definition we went straight to the wireframes directly on Figma.

- **Wireframes and UI design**: Designed using Figma, these mapped the UI structure and nqvigqtion



The above diagrams show the figma design of the MFix mobile applications. Starting with the welcome page, the home page, diagnosis pages(dashboard and sound diagnosis respectively), profile page and requests mechanic page.

3.4. GLOBAL ARCHITECTURE OF THE SOLUTION

System architecture is an essential aspect of any software system, as it determines how the different components are structured and how they interact. For the MFix car fault diagnostic system, a **multi-layered architecture** was adopted. This architectural approach promotes scalability, maintainability, and efficient separation of concerns, making it ideal for complex systems like MFix.

The multi-layered architecture divides the system into distinct, manageable parts:

- **Presentation Layer (Frontend)**: Implemented using React Native and Expo, this layer handles all user interactions, navigation, and visual rendering.
- **Business Logic Layer (Backend)**: Powered by Node.js and Express.js, this layer manages all processing logic, routing, API definitions, and security controls.
- **Data Access Layer (Firebase Admin SDK)**: Connects the backend to Firestore, handling data reads, writes, and validations through Firebase's APIs.
- **Data Layer (Firebase Firestore)**: A scalable NoSQL cloud database storing all entities such as users, vehicles, diagnostics, and results.

In addition to the multi-layered approach, the **MVVM (Model–View–ViewModel)** architectural design pattern was chosen for the frontend structure. MVVM provides a clear abstraction between the UI and the underlying logic, which is highly beneficial for building maintainable and scalable mobile applications.

Justification for Choosing MVVM:

- **Data Binding**: MVVM simplifies the binding of complex diagnostic data from the ViewModel directly to the View, making the UI more responsive and dynamic.
- **Separation of Concerns**: It enforces a strict separation between data management (Model), application logic (ViewModel), and the user interface (View), enhancing modularity and code clarity.

- **Flexibility and Testability**: The pattern allows easy integration with different data sources and facilitates unit testing, as the ViewModel can be tested independently of the UI.

This architectural choice ensures that the MFix system remains extensible and adaptable to future upgrades such as hardware integration (e.g., OBD-II), machine learning-based diagnostics, or offline functionality.

Overall, the chosen architecture combines **robust layering principles** with the **MVVM pattern**, offering a modern, flexible, and maintainable foundation for the MFix diagnostic platform.

3.5.DESCRIPTION OF THE RESOLUTION PROCESS

The problem was tackled in stages:

1. **Requirement Analysis**:
    - Stakeholder interviews to determine features (diagnostics, request tracking, notifications).
    - Prioritized features based on feasibility.
2. **System Modeling and UI Design**:
    - Determining app Identity and brand identity then created wireframes in Figma.
    - Designed Firestore schema with collections and relationships.
3. **Frontend Development**:
    - Built UI using React Native with reusable components.
    - Integrated navigation and persistent state management.
4. **Backend Development**:
    - Developed RESTful APIs using Node.js and Express.
    - Configured Firebase Admin SDK for database operations.
5. **Database Integration**:
    - Firestore collections were created to match schema.
    - Backend routes accessed data using Firebase Admin.

6. **Testing and Mocking**:
    - Used mock data and simulated diagnostics using setTimeout.
    - Postman tested APIs; UI tested on Expo Go.
7. **Deployment Preparation**:
    - Expo CLI builds for Android devices.
    - Firebase security rules tested in staging.

3.6.PARTIAL CONCLUSION

The analysis and design phase laid a strong foundation for the MFix mobile diagnostic system. Through structured modeling, system decomposition, and iterative wireframing, we ensured the solution would meet both functional and non-functional requirements. The architecture chosen supports modular development, real-time interaction, and future AI integration, making it suitable for deployment in real-world automotive environments. The next chapter transitions into the full implementation and validation of the proposed solution.

4. CHAPTER 4: IMPLEMENTATION AND RESULT

4.1.INTRODUCTION

In this chapter, we present the detailed implementation process of the MFix car fault diagnostics application, including the tools and materials used, the step-by-step development process, and the results obtained. We also explain how to deploy and run the application, followed by an evaluation of the solution compared to existing diagnostic methods. The chapter concludes with a summary of the implementation outcomes.

4.2. TOOLS AND MATERIAL USED

4.2.1. Frontend Development Technologies and Frameworks

- **React Native (with Expo):** Cross-platform framework used for building the mobile app with native performance and web-like development efficiency.
- **JavaScript (ES6+):** Primary programming language for frontend and backend logic.
- **React Navigation:** Routing and navigation management library for smooth transitions across app screens.
- **React Context API:** Global state management to maintain session data, user roles, and diagnostics results.
- **AsyncStorage:** For lightweight local persistent storage of tokens and preferences.
- **Figma:** UI/UX design tool used to create high-fidelity app prototypes and wireframes.
- **React Native Maps:** Provides interactive map functionality within the app interface.

4.2.2. Backend and Database Technologies

- **Node.js:** JavaScript runtime environment for backend services.
- **Express.js:** Web framework for creating RESTful API endpoints.
- **Firebase Cloud Firestore:** NoSQL cloud database for real-time data synchronization, scalable storage, and secure access management.
- **Firebase Admin SDK:** Backend service integration with Firebase Authentication, Firestore, and Storage.
- **dotenv:** Module to securely manage environment variables.
- **Postman:** API testing tool for endpoint verification.

4.2.3. AI/ML Tools

- **HuggingFace Pre-trained Audio Classification Models:** Used as a base for finetuning engine sound recognition.
- **Kaggle Datasets:** Source of labeled engine sound recordings used to train the initial sound diagnostic model.

4.2.4. Development and Testing Tools

- **Visual Studio Code:** IDE for coding.
- **Android Emulator / iOS Simulator:** For testing app on multiple platforms.
- **Git/GitHub:** Version control and collaboration.
- **Expo Go:** To test builds easily on physical devices.
- **Postman:** API testing tool for endpoint verification.

4.3.DESCRIPTION OF IMPLEMENTATION PROCESS

This section outlines the step-by-step development of the MFix application, covering the implementation of the frontend, backend, and database components. It details how these parts were designed, built, and integrated to create a seamless, real-time vehicle diagnostics system. Additionally, it discusses the approaches taken for navigation, state management, and testing to ensure a robust and user-friendly application.

4.3.1. Frontend Implementation

- We developed the mobile app UI using **React Native** with Expo, focusing on responsiveness and fidelity to the Figma design.
- Implemented key screens such as Welcome, Home Dashboard, Diagnostics Input, Fault Analysis, History, and Verification.
- Used **React Navigation** (Stack and Tab navigators) for seamless user flow.
- Managed global app state via **React Context API** to store user session info, roles, and diagnostic data.
- Employed **AsyncStorage** for persistent data storage like login tokens.
- Designed UI components to be modular, reusable, and adaptable across different screen sizes using flexbox and percentage-based dimensions.

- Incorporated placeholder components for future AI-powered image and sound diagnostics.
- Prepared the frontend for backend integration by creating mock API calls with simulated delays to test lazy loading and error handling.

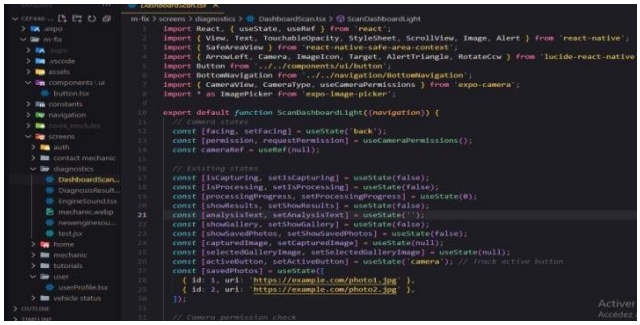


Fig 15: A piece of code for scanning dashboard

4.3.2. Database Implementation

- Selected **Firebase Cloud Firestore** for its flexibility, real-time sync, and minimal server maintenance.
- Designed a logical, scalable NoSQL schema with collections for:

Users: storing user profile and roles.

- **Vehicles:** referencing users and vehicle info.
- **Mechanics:** with availability and rating info.
- **Diagnostics:** storing diagnosis sessions.
- **Results:** detailed fault analysis, explanations, and recommendations.
- **Requests:** user-to-mechanic service requests.
- **Notifications:** user alerts and messages.

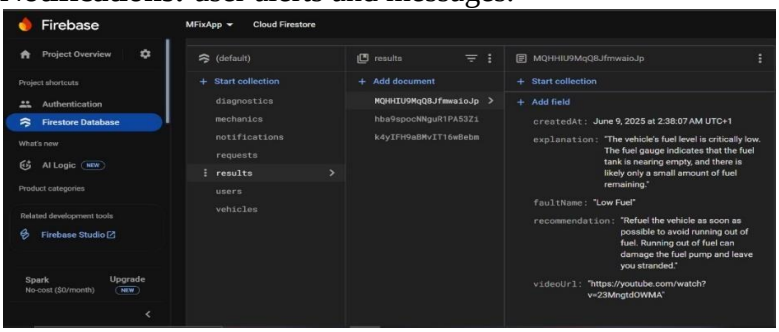


Fig 16 : Firebase Overview.

- Enforced constraints and relationships logically using:
  - Frontend validation.
  - Firestore Security Rules.
  - Firebase Cloud Functions for business logic.
- Created composite indexes on key fields (e.g., userId + createdAt) to optimize query performance.
- Applied denormalization and pagination to reduce reads and improve responsiveness.

4.3.3. Backend Implementation

- Implemented backend using **Node.js** and **Express.js** for RESTful API creation.
- Integrated **Firebase Admin SDK** for secure server-side operations with Firestore and Authentication.
- Developed API routes to support core features:
  - User registration and fetching.
  - Vehicle data management.
  - Image and sound diagnostics analysis endpoints.
  - Result retrieval.
  - Service requests and notifications handling.

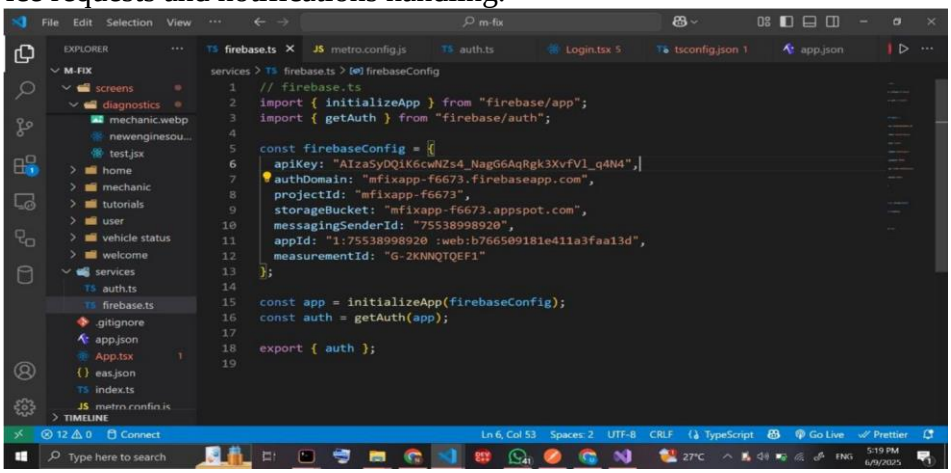


Fig 17: Firebase configuration for authentication

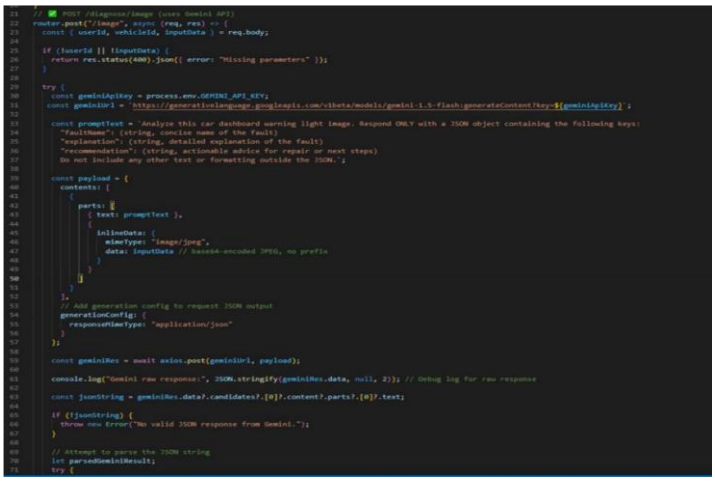


Fig18: Image diagnosis code

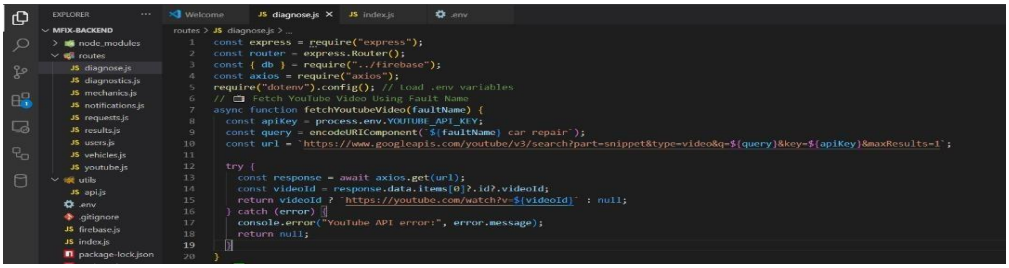


Fig19: Integrationg the Youtube API

- Handled authentication, validation, error logging, and response status codes to ensure robust backend communication.
- For sound diagnostics, fine-tuned an audio classification model based on a Kaggle dataset and HuggingFace pre-trained models, laying groundwork for future AI integration.

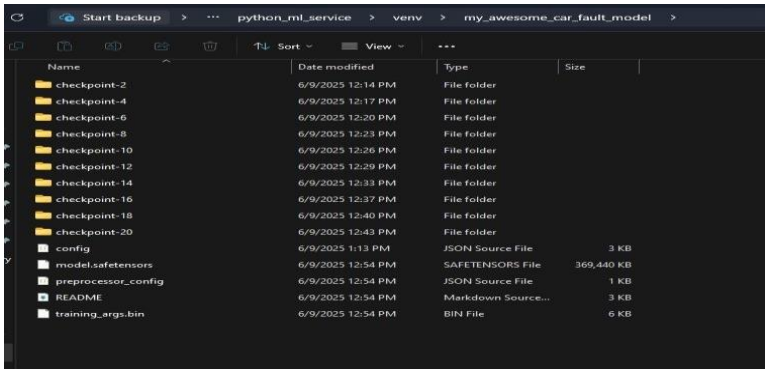


Fig 20: training and organising model for sound recognition

4.3.4. Database Integration with Backend

Before the backend can interact with Firestore, Firebase must be properly configured using service credentials. In MFix, we created a Firebase project and downloaded the serviceAccountKey.json file containing private keys and identifiers. This file was securely loaded using Node's fs module.

 Connection Logic in Backend

The backend uses the Firebase Admin SDK to establish a persistent connection to Firestore. This connection is initiated once, globally, in index.js and shared across routes using the exported db reference:

```
// Shared Firestore instance const
db = admin.firestore();
module.exports = db;
```

Each backend route (e.g., users, diagnostics) imports the database instance and performs operations using Firestore's document and collection APIs.

Example in **diagnostics file:**

```
const db = require("../config/firebase"); router.post('/image',

async (req, res) => {


  const result = await db.collection('results').add({...});
res.status(200).json({ resultId: result.id });

});
```

 Error Handling and Logging

To ensure stability and ease of debugging, all Firestore operations are wrapped in try-catch blocks. Errors are logged to the console and optionally stored or sent to a monitoring service in future versions.

Example:

```
try {
  const data = await db.collection('vehicles').doc(vehicleId).get();
res.json(data.data()); } catch (error) {  console.error("Error
fetching vehicle:", error.message);  res.status(500).json({ error:
"Internal server error" });
}
```

We used status codes like 400, 404, and 500 to return meaningful messages to the frontend for better user feedback and UI handling.

4.3.5. Integration with Frontend

- Connected frontend React Native app with backend RESTful APIs via HTTP requests.
- Used mock API functions during frontend development for testing dynamic data loading and error states.
- Designed all UI components to consume backend data structures with minimal changes once live APIs are connected.
- Prepared the frontend to handle asynchronous data, lazy loading, and real-time updates when integrated with Firestore.

4.3.6. Navigation and State Management

- Implemented **React Navigation** stacks and tab navigators for structured user journeys.
- Used **React Context API** globally to manage user authentication state, roles, and diagnostics data across screens.
- Ensured that user session data and selections persist across navigation events without data loss.

- Maintained scalability by separating concerns in navigation and state logic.

4.3.7. Testing and Deployment

- Performed manual testing on multiple devices and simulators to verify:
- UI responsiveness and consistency with design.
- Navigation flow and state persistence.
- Backend API response handling with Postman.

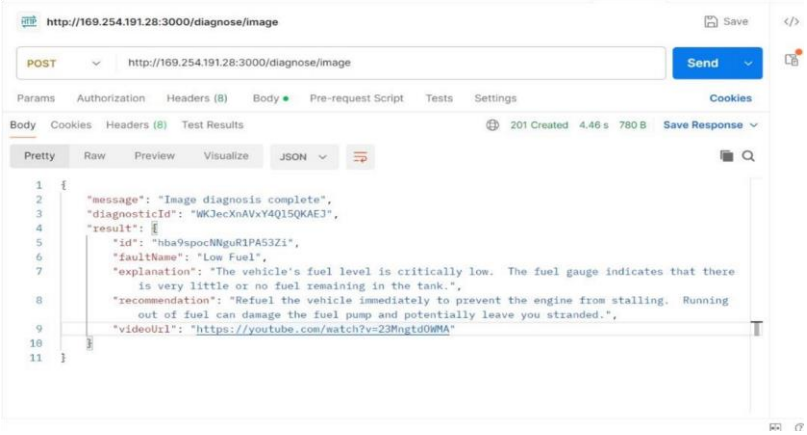


Fig 21: Test response in postman

- Error handling and loading states on the frontend.
- Identified and resolved issues related to asynchronous data fetching, user input validation, and UI layout adaptation.

4.4. Presentation and Interpretation of Results

After implementing the core components of the MFix system, the application was run on both Android and iOS devices using Expo. Below are the key screens of the app, along with a brief explanation of their functionality and the results observed.

4.4.1. Welcome Screen

The welcome screen introduces users to the app and provides access to login or register. It ensures a consistent visual identity with the MFix brand, using the selected color palette and typography.

Result: Clean and professional interface that directs the user clearly into the app flow.

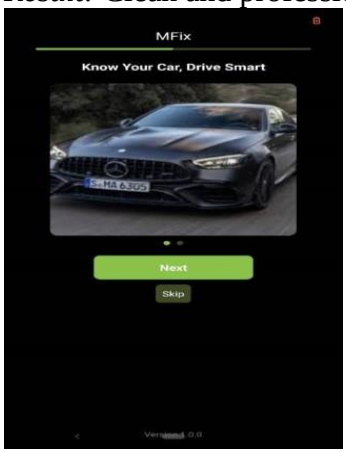


Fig 22: Welcome screen

4.4.2. Home Dashboard

Once logged in, users are taken to the dashboard which displays summary info like vehicle status, pending requests, and a quick-access menu.

Result: The dashboard consolidates core functionalities and presents a clear overview, enabling users to take quick actions.

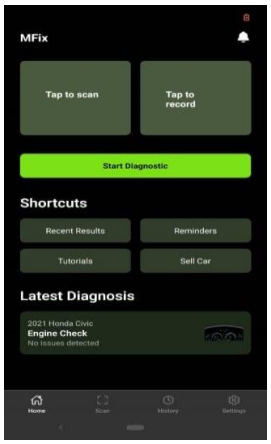


Fig 23: Home Dashboard screen

4.4.3. Diagnostic Screen

This screen allows users to simulate a diagnostic. Users can either manually enter an issue or simulate image/sound upload.

Result: Diagnostics return results based on predefined fault codes. Results are presented with fault names, explanations, recommendations, and optionally a video link.

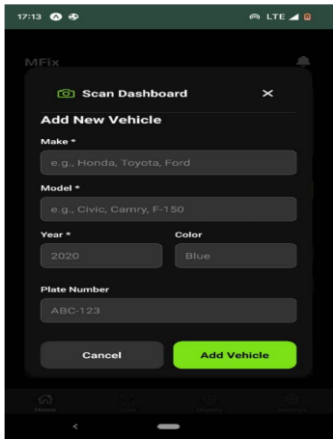


Fig 24: dashboard scanning

Fig 25: Sound Recording

Scan or upload dashboard photo

Record engine sound

4.4.4. Diagnosis Result Page

Detailed view of a diagnostic report including the result type (sound/image), status, analysis summary, and recommendations.

Result: Successfully displays mock data in structured format, ready for API integration.

Fig26: Processing Image

4.4.5. Vehicle Registration Screen

Users can register a vehicle by entering make, model, and year.

Result: Form validation was successful; data was correctly formatted and sent to the backend (or stored as mock data).

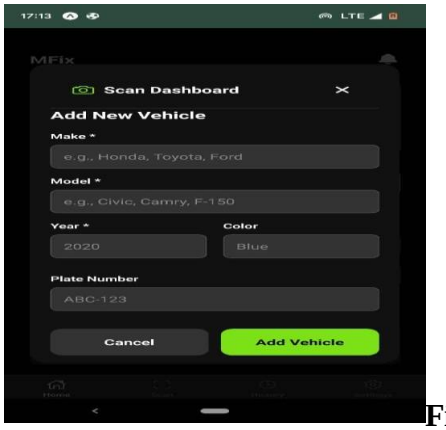


Fig28: Vehicle registration

4.4.6. Requests and Notifications

Users can view their diagnostic requests sent to mechanics and any notifications regarding updates.

Result: Efficient structure for listing and managing interactions with mechanics. Notifications display real-time (mocked) alerts.

Fig29: Requests mechanics

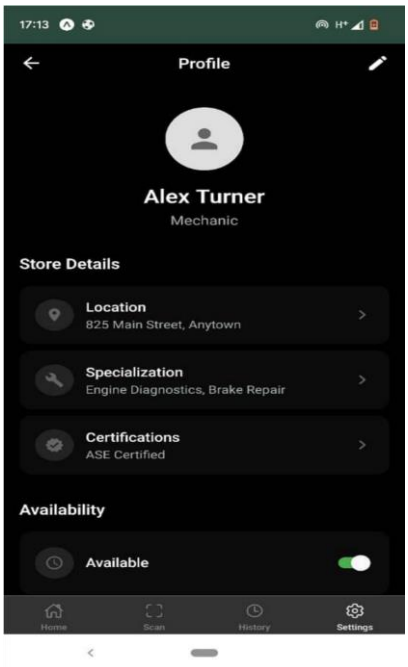
4.4.7. Search for nearby mechanics

Users can check nearby mechanics based on their location

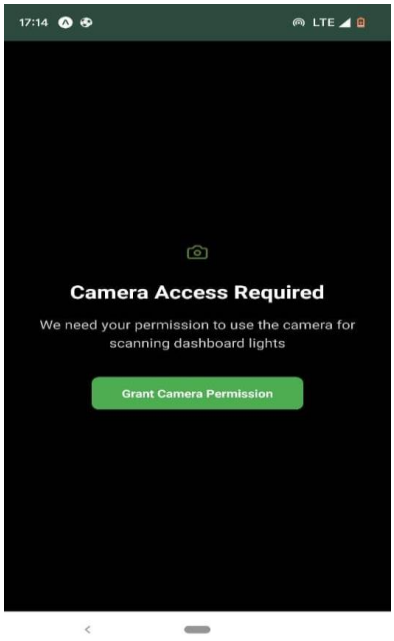
Result: clean design and structure showing closest mechanics and a map.

Other pages include

Login/registration



Profile page



camera permission page

4.4.8. Interpretation of Results

- **Responsiveness:** The UI adapts well to various device screen sizes, maintaining clarity and functionality.

- **Performance:** The app loads quickly, even with mock data, and transitions smoothly between screens.
- **User Flow:** Navigation is intuitive, and the separation between roles (user vs. mechanic) enhances the user experience.
- **Preparedness:** All screens are structurally and functionally ready for backend API integration, with placeholders where real-time data will be injected.

4.5. EVALUATION OF THE SOLUTION

Here, we focused on evaluating our objectives. Evaluating our objectives will help us determine how good our solution is.

Objective	**Achieved?**	**Justification**
Develop a mobile car diagnostics tool	✓	App is functional and deployable via Expo
Provide multimedia-based diagnostics	medium	Image/sound structure in place, basic model trained
Enable real-time updates	✓	Firestore provides real-time sync and updates
Design for future scalability and AI	✓	Modular structure and APIs support future growth

4.6.PARTIAL CONCLUSION

The implementation of the MFix system marks a significant step forward in the modernization of vehicle diagnostics, especially in mobile and resource-constrained environments. By combining a cross-platform frontend, a cloud-based backend, and a scalable real-time database, MFix successfully lays the groundwork for intelligent, user-centered vehicle maintenance.

Through this chapter, we have detailed the tools used, the full-stack development process, and the results obtained. The app demonstrates key functionalities including user registration, rolebased access, fault diagnostics (via mock data), mechanic interaction, and notification handling all structured for seamless backend integration and future AI enhancement.

While current limitations exist, such as the use of mock data and basic AI models, the architectural decisions made ensure that the system is extensible and ready for production-level deployment. The evaluation has shown that MFix satisfies its major objectives and contributes meaningfully to the fields of mobile computing, automotive technology, and cloud-based diagnostics

5. CHAPTER 5: CONCLUSION AND FURTHER WORKS

5.1. Summary of Findings

This project set out to design and implement **MFix**, a mobile-based vehicle fault diagnostics application that leverages modern technologies such as React Native, Firebase, and AIreadiness to provide an intelligent, scalable, and real-time solution for car fault detection.

From the research and implementation phases, the following findings emerged:

- Mobile applications can significantly enhance vehicle diagnostics accessibility, especially when built on flexible and lightweight frameworks like React Native.
- Firebase Firestore proved effective in managing real-time, scalable data operations without the burden of server management.
- A multi-role design (user and mechanic) allows tailored user experiences and broadens system applicability.
- The integration of multimedia diagnostics (image and sound) is feasible and holds great potential, even though current implementations use mock data or basic AI models.
- React Context API and Firebase authentication simplify state and session management while preserving security and performance.

These findings validate that MFix is a promising alternative to traditional OBD-II tools, providing a foundation for smarter diagnostics, especially in developing environments.

5.2. Contribution to Engineering and Technology

MFix contributes to the fields of **automotive engineering, mobile computing**, and **cloudbased diagnostic systems** in the following ways:

- **Enhanced Accessibility:** By shifting diagnostics from hardware tools to a smartphone app, MFix reduces reliance on physical OBD-II scanners, opening up diagnostics to users with minimal equipment.
- **Cloud Integration:** Real-time data handling via Firestore allows for immediate updates, collaboration, and request tracking across users and mechanics.
- **AI-Ready Architecture:** Although preliminary, the inclusion of sound-based diagnostics via a fine-tuned audio classification model demonstrates how machine learning can enhance automotive fault detection.
- **User-Mechanic Ecosystem:** Introducing a role-based service model, MFix supports request tracking and professional feedback loops, unlike most current apps which are user-only.
- **Scalable Design:** The architecture supports future expansion, including integration with IoT vehicle sensors, live diagnostics, and admin portals.

Compared to existing diagnostic systems that are either hardware-dependent or limited in scope, MFix bridges the gap between **usability**, **intelligence**, and **mobility**, marking a step forward in engineering innovation.

5.3. Recommendations

To further develop and deploy MFix at scale, the following recommendations are made:

1. **Integrate Real OBD-II APIs:** Use Bluetooth or Wi-Fi OBD-II adapters to fetch live vehicle data and enable real-time, data-driven diagnostics.

2. **Expand AI Capabilities:** Improve the current sound analysis model with a larger, more diverse dataset, and incorporate computer vision for dashboard error light recognition.
3. **Develop a Web-Based Admin Portal:** Introduce an admin layer for managing users, analyzing data trends, and generating performance reports.
4. **Develop the mechanic side of the app.**
5. **Ensure Offline Capabilities:** Implement offline caching for areas with low connectivity.
6. **Security Hardening:** Introduce two-factor authentication (2FA), encrypted storage, and role-based access controls to enhance data protection.
7. **Automate Testing & CI/CD:** Adopt continuous integration pipelines to automate testing, deployment, and updates.

5.4. Difficulties Encountered

The MFix project development encountered a number of technical and logistical challenges, including:

- **Hardware Constraints:** Limited development resources forced a switch from Flutter to React Native due to system performance issues.
- **AI Training Dataset Limitations:** The dataset used for sound diagnostics was relatively small, which constrained model accuracy and generalization.
- **Limited Testing Environment:** Testing was primarily done on simulators and a few physical devices, restricting insight into performance on low-end or diverse real-world phones.
- **Internet Dependency:** The current version relies on active internet connection due to Firebase integration.

Despite these limitations, workarounds were employed, such as progressive design techniques, and modular architecture to ensure future adaptability.

5.5. Further Works

To build upon the current success of MFix, the following future developments are suggested

- **Advanced Machine Learning Models:** Train and deploy a more robust ML pipeline using tools like TensorFlow Lite or CoreML for on-device fault detection.
- **IoT Integration:** Connect to car sensors or external diagnostic devices for live fault reading and streaming to the app.
- **Admin Analytics Dashboard:** Provide an analytics portal to monitor diagnostic trends, mechanic efficiency, and user behavior.
- **Multi-Language Support:** Add language localization to make the app accessible in diverse geographic regions.
- **Role based access.**
-
- **Real-Time Mechanic Locator:** Integrate GPS-based mechanic search and appointment booking for improved service response time.

Through the realization of the MFix project, we have taken a significant step toward digitizing and democratizing vehicle diagnostics. By embracing mobile technologies, cloud platforms, and AI principles, MFix not only addresses current gaps in fault detection tools but also lays the groundwork for intelligent automotive systems of the future.

REFERENCES

1. Google. (n.d.). *Cloud Firestore documentation*. Firebase. https://firebase.google.com/docs/firestore
2. Meta Platforms, Inc. (n.d.). *React Native documentation*. https://reactnative.dev/docs/getting-started

3. Expo.dev. (n.d.). *Expo documentation*. https://docs.expo.dev/
4. Mozilla Developer Network. (n.d.). *JavaScript (JS) documentation*. https://developer.mozilla.org/en-US/docs/Web/JavaScript
5. Node.js Foundation. (n.d.). *Node.js documentation*. https://nodejs.org/en/docs
6. Express.js. (n.d.). *Express - Node.js web application framework*. https://expressjs.com/
7. Postman. (n.d.). *API platform for building and using APIs*. https://www.postman.com/
8. Hugging Face. (n.d.). *Pre-trained models for audio classification*. https://huggingface.co/models
9. Kaggle. (n.d.). *Engine sound classification dataset*. https://www.kaggle.com/
10. W3Schools. (n.d.). *MVVM architecture pattern*. https://www.w3schools.blog/mvvmarchitecture
11. Microsoft. (n.d.). *MVVM Design Pattern*. https://learn.microsoft.com/enus/archive/msdn-magazine/2009-february/mvvm-design-pattern
12. Boudreau, D. (2020). *Mobile App Architecture: Guide for Developers*. Medium. https://medium.com
13. Firebase. (n.d.). *Firebase Admin SDK*. https://firebase.google.com/docs/admin/setup
14. GitHub. (n.d.). *Open-source diagnostic apps and Firebase integrations*. https://github.com/

APPENDICES

1. **Appendix A:** Survey Forms and Questionnaires
   - *Content:* Raw data from user surveys conducted during requirements gathering.
2. **Appendix B:** Full Code Snippets
   - *Content:* Extended code examples for key functionalities (e.g., Firebase setup, API routes).
3. **Appendix C:** Dataset Details
   - *Content:* Description of the Kaggle dataset used for sound model training.
4. **Appendix D:** Testing Reports
   - *Content:* Detailed results from unit, integration, and user testing.

5. **Appendix E:** UI Wireframes (Figma Prototypes)

 Content: High-fidelity mockups of all app screens.

6. **Appendix F:** Firestore Security Rules

 Content: Configuration for database access control.

7. **Appendix G:** Project Timeline and Milestones

 Content: Gantt chart or Agile sprint summaries.